

PENETRATION TESTING FUNDAMENTALS

MODULE 6

Exploiting Application-Based Vulnerabilities & OWASP Top 10

Student Study Guide

Topics: HTTP Protocol · URL Structure · Web Sessions · OWASP · Business Logic · SQL Injection · XSS · CSRF · IDOR · Session Hijacking · Clickjacking · Directory Traversal · File Inclusion · API Security · Code Signing · Web Hacking Tools

1. The HTTP Protocol

Web applications use many different protocols, the most prevalent of which is HTTP. HTTP 1.1 is defined in RFCs 7230–7235. Understanding HTTP deeply is essential for web application penetration testing — both for identifying vulnerabilities and for crafting exploits.

Key HTTP Facts

- HTTP is a stateless protocol — each request/response transaction is independent with no memory of prior interactions
- An HTTP transaction = a single request from a client to a server, followed by a single response
- Stateful protocols (FTP, SMTP, IMAP, POP) keep track of previous interactions and maintain session state across commands
- HTTP clients include: web browsers, proxies, API clients, and custom HTTP programs
- HTTP proxies act as both servers and clients — enabling transfers across firewalls, caching, NAT, and HTTP request filtering

NOTE

When a protocol is stateful, it keeps track of previous interactions — maintaining the state of the conversation/session. A sequence of transmitted and executed commands is often called a session.

1.1 HTTP Status Code Ranges

Range	Category	Common Examples
1xx	Informational	100 Continue, 101 Switching Protocols
2xx	Successful	200 OK, 201 Created, 204 No Content
3xx	Redirection	301 Moved Permanently, 302 Found, 304 Not Modified
4xx	Client errors	400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found
5xx	Server errors	500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable



TIP

Download Wireshark and capture traffic between your browser and a web server. Analysing real HTTP traffic is one of the best ways to understand how web applications communicate. Also use curl with --http2 to see differences between HTTP/1.1 and HTTP/2.

1.2 HTTP URL Structure

Every component of a URL is a potential attack surface. Understanding each part is essential for identifying and exploiting web vulnerabilities. Consider this example:

```
https://theartofhacking.org:8123/dir/test;id=89?name=omar&x=true
```

Component	Example Value	Description
Scheme	https	The underlying protocol (HTTP, HTTPS, FTP) — followed by ://
Host	theartofhacking.org	IP address or DNS name of the web server being accessed
Port	8123	Optional port number (default: 80 for HTTP, 443 for HTTPS)
Path	/dir/test	Path from server root to the desired resource — may use aliasing
Path-segment params	id=89	Optional name/value pairs preceded by semicolon — not commonly used but sometimes attackable
Query string	name=omar&x=true	Dynamic parameters — preceded by ?, name=value pairs separated by &



NOTE

REST APIs use JSON as the standard output format. SOAP is an older XML-based API protocol used in legacy applications. XML-RPC is another legacy XML-over-HTTP protocol. In pen testing, all these API formats present unique and different attack surfaces.

2. Web Sessions & Session Management

A web session is a sequence of HTTP request and response transactions between a web client and a server. It includes pre-authentication tasks, the authentication process, session management, access control, and session termination. Applications use session IDs (tokens) to track users. Once an authenticated session is established, the session ID is temporarily equivalent to the strongest authentication credential used by the application.

2.1 Session ID Fingerprinting

The session ID name used by a web framework immediately reveals the technology stack — useful intelligence for an attacker:

Session ID Name	Technology Stack
PHPSESSID	PHP
JSESSIONID	Java EE (J2EE)
CFID / CFTOKEN	Adobe ColdFusion
ASP.NET_SessionId	ASP.NET / Microsoft

**TIP**

Best practice: Rename the default session ID to a generic name such as 'id' to prevent framework fingerprinting.

2.2 Session ID Security Requirements

- Must be at least 128 bits (16 bytes) — anything smaller is vulnerable to brute force
- Must be unique and unpredictable — use a DRBG (Deterministic Random Bit Generator) providing at least 256 bits of entropy
- Must NEVER appear in the URL — include it only in cookies (prevents session fixation and URL leakage)
- Must be transmitted ONLY over encrypted (HTTPS) channels — for the entire session, not just login
- Use the HTTPOnly cookie flag — prevents client-side JavaScript from accessing the cookie (protects against XSS-based session theft)

- Use non-persistent (session) cookies — session ID is deleted when the browser closes
- Two types of cookies: non-persistent (session cookies) and persistent (with Max-Age or Expires attribute stored to disk)

IMPORTANT

Session fixation attacks occur when an attacker intercepts web traffic and injects (fixes) a known session ID on the victim's browser before authentication. The attacker can then use that same session ID to authenticate as the victim. Enforcing HTTPS for the entire session — not just the login page — prevents this.

3. OWASP & Building a Web Application Lab

The Open Web Application Security Project (OWASP) is an international organisation dedicated to educating professionals and creating tools and best practices for securing web applications. OWASP publishes and regularly updates the OWASP Top 10 — an awareness document identifying the most critical web application security risks. Reference: owasp.org

KEY CONCEPT

The OWASP Top 10 is an awareness document — not a compliance checklist. It identifies the most prevalent and impactful categories of web vulnerabilities. Every security professional must know it thoroughly.

3.1 Lab Environment Options

Distribution	Base OS	Download
Kali Linux	Debian — most popular, maintained by Offensive Security	kali.org
Parrot OS	Debian-based alternative, suitable for bare metal and VMs	parrotsec.org
BlackArch Linux	Arch Linux — 1,900+ tools and packages	blackarch.org

TIP

For beginners: Install Kali Linux or Parrot OS in VirtualBox or VMware, then set up WebSploit Labs ([websploit.org](https://www.websploit.org)) for a safe, intentionally vulnerable web application environment. Omar Santos GitHub repository (h4cker.org/github) contains hundreds of resources, vulnerable applications, and Docker containers for practice.

4. Business Logic Flaws (CWE-840)



Business Logic Flaws

Business logic flaws enable attackers to use legitimate application transactions and flows in ways that produce unintended negative outcomes. Unlike XSS, SQLi, and CSRF, business logic flaws cannot typically be found using automated scanners — they require understanding the application's intended behaviour and abusing it.

CWE-840 Examples (MITRE)

- Unverified ownership — accessing resources belonging to other users without authorisation
- Authentication bypass using an alternate path or channel
- Authorisation bypass through user-controlled key
- Weak password recovery mechanism for forgotten passwords
- Incorrect ownership assignment
- Allocation of resources without limits or throttling (rate limit bypass)
- Premature release of resource during expected lifetime
- Improper enforcement of a single, unique action
- Improper enforcement of a behavioural workflow



NOTE

Data validation and a detailed threat model are the primary defences against business logic flaws. OWASP testing guide: owasp.org/www-project-web-security-testing-guide

5. Injection-Based Vulnerabilities

Injection vulnerabilities occur when an attacker sends untrusted data to an interpreter as part of a command or query. Successful injection attacks can lead to data disclosure, data manipulation, denial-of-service, and full system compromise. Types covered in this module: SQL injection, command injection, LDAP injection, and HTML injection.

5.1 SQL Injection (SQLi)

SQL Injection (CWE-89)

SQL injection allows an attacker to view, insert, delete, or modify records in a database by injecting partial or complete SQL queries via web application input fields or URL parameters. SQL injection can be catastrophic — databases typically contain an organisation's most sensitive and business-critical data.

Essential SQL Commands for Pen Testers

Command	Purpose
SELECT	Retrieve data from a database
INSERT INTO	Insert new data into a database
UPDATE	Modify existing data in a database
DELETE	Remove records from a database
DROP TABLE	Delete an entire table — catastrophic if injected
CREATE / ALTER	Create or modify database objects
UNION	Combine results of two or more SELECT statements — key technique in SQLi

TIP

Start testing for SQLi by adding a single quote (') or semicolon (;) to input fields and URL parameters. The single quote is a SQL string terminator — if unfiltered, it may generate a database error or return unexpected data. Also try: --, /* */ (comment delimiters), AND, OR operators.

SQL Statement Categories

- **DDL (Data Definition Language):** CREATE, ALTER, DROP — define and modify database structure
- **DML (Data Manipulation Language):** SELECT, INSERT, UPDATE, DELETE — manipulate data

- **Transaction Control:** COMMIT, ROLLBACK, SAVEPOINT — manage database transactions
- **Session / System Control:** Control database sessions and system-level settings

SQL Injection Categories

Category	How It Works	Detection Difficulty
In-band SQLi	Data is retrieved via the same channel used to inject — results appear directly in the web response. Most basic form of SQLi.	Easiest — errors or data visible in the response
Out-of-band SQLi	Results are sent via a different channel — DNS, HTTP request to attacker's server, email. Used when in-band channels are blocked.	Medium — requires attacker-controlled infrastructure
Blind (Inferential)	No data returned in the response — attacker infers information from application behaviour differences (Boolean) or response timing.	Hardest — requires automated tools and patience

SQL Injection Exploitation Techniques

1. **UNION Operator:** Combines the results of two SELECT statements. Goal: extract columns from other database tables. Both SELECT statements must return the same number of columns.
2. **Boolean Technique:** Inject TRUE/FALSE SQL conditions and observe differences in application behaviour (page content, HTTP codes, redirects) to infer database content.
3. **Error-Based Technique:** Force the database to generate an error that reveals the database type, version, table, or column structure.
4. **Out-of-Band Technique:** Use DBMS functions (xp_cmdshell, load_file, LOAD_FILE, UTL_HTTP) to send query results to an attacker-controlled server via HTTP or DNS.
5. **Time Delay:** Inject sleep/delay commands — a delayed response confirms a TRUE condition in blind SQLi without any visible data in the response.

UNION Exploitation Example

```
-- Original legitimate query:
SELECT zipcode FROM h4cker_customers WHERE zip=1

-- Attacker appends UNION to extract credit card data:
SELECT zipcode FROM h4cker_customers WHERE zip=1
UNION ALL SELECT creditcard FROM payments

-- Result: the page now returns credit card numbers alongside zipcodes
```


Stacked Queries

A semicolon (;) ends one SQL statement and begins another. Stacked queries allow executing any SQL statement — not just SELECT (like UNION). This enables DELETE, DROP, and INSERT attacks:

```
-- Malicious input provided by attacker:
1; DELETE FROM customers

-- Full query executed by the vulnerable application:
SELECT * FROM customers WHERE customer_id=1; DELETE FROM customers

-- Result: ALL customer records deleted from the database
```

Time Delay Technique (MySQL Example)

```
https://store.h4cker.org/buy.php?id=8
AND IF(version() like '8%', sleep(10), 'false'))--

-- If MySQL version starts with '8', server delays response by 10 seconds
-- A delay confirms the condition is TRUE
-- The attacker can enumerate database version, structure, and data this way
```

Stored Procedure Injection

A stored procedure is pre-compiled SQL on the server. If stored procedures do not sanitise their input, they are also vulnerable to injection:

```
-- Vulnerable stored procedure:
Create procedure user_login @username varchar(20), @passwd varchar(20)
As Declare @sqlstring varchar(250)
Set @sqlstring = 'Select 1 from users Where username = '
                + @username + ' and passwd = ' + @passwd
exec(@sqlstring)

-- Attacker input: username = omar' or 1=1--
-- This bypasses authentication entirely
```

Database Fingerprinting

To execute advanced SQLi attacks, first identify the backend database. The easiest method is to observe error messages — each database vendor returns distinctly formatted error codes. Databases differ in syntax, functions, and available features — what works in MySQL may not work in Oracle or MSSQL.



MySQL error reference: dev.mysql.com/doc/refman/8.0/en/error-handling.html

MSSQL error reference: learn.microsoft.com/en-us/sql/relational-databases/errors-events

Oracle error reference: docs.oracle.com/database/121/ERRMG/toc.htm

SQL Injection Mitigations

✓ REMEMBER

The best defence against SQL injection is immutable queries — queries that cannot be altered by user input. NEVER concatenate user-supplied strings directly into SQL queries.

- **Parameterised queries (prepared statements):** User input is treated as data only — never interpreted as SQL code. This is the most reliable defence.
- **Stored procedures (immutable):** Pre-compiled, pre-validated SQL — safe only if they do not themselves generate dynamic SQL internally.
- **Static queries:** Hard-coded queries with no user-supplied components — maximum safety but inflexible.
- **Input validation (allow-listing):** Accept only expected character types and lengths. Reject everything else.
- **OWASP ESAPI:** Open-source web security control library providing SQLi, XSS, and CSRF mitigations.

```
// Java parameterised query — the correct approach:
String query = "SELECT * FROM users WHERE name = ?";
PreparedStatement stmt = connection.prepareStatement(query);
stmt.setString(1, username); // Input treated as DATA — never parsed as SQL
ResultSet results = stmt.executeQuery();

// Static query examples (simplest form):
select * from contacts;
select * from users where user = "omar";
```

🔧 TOOLS

SQLmap — automated SQL injection detection and exploitation. Pre-installed in Kali Linux and Parrot OS. Download: sqlmap.org

WebGoat / WebSploit Labs — intentionally vulnerable apps for SQLi practice in a safe environment.

5.2 Command Injection

Command Injection

A command injection attack involves executing operating system commands on the target server via a vulnerable web application. This is possible when an application passes unsanitised user input (from web forms, cookies, HTTP headers) directly to a system shell. Injected commands run with the privileges of the vulnerable web application process.

Note: Command injection is NOT the same as code execution via buffer overflow — it exploits input validation failures in application logic, not memory corruption vulnerabilities.

NOTE

Modern web application frameworks have built-in defences that have significantly reduced the prevalence of command injection. However, older custom applications and scripts remain vulnerable. OWASP: owasp.org/index.php/Command_Injection

5.3 LDAP Injection

LDAP Injection

LDAP injection attacks leverage input validation failures to inject queries directly into LDAP servers. LDAP (Lightweight Directory Access Protocol, defined in RFC 4511) is used by organisations to access and maintain directory services. An attacker who exploits LDAP injection can bypass authentication, enumerate directory resources, or modify the LDAP directory tree.

LDAP Injection Type	What the Attacker Can Do
Authentication bypass	Most basic LDAP injection — bypass password and credential checking entirely without valid credentials
Information disclosure	Inject crafted LDAP packets to list all resources in the organisation's directory — used for reconnaissance

6. Exploiting Authentication-Based Vulnerabilities

Authentication vulnerabilities allow attackers to access accounts, data, or functionality without providing valid credentials. The most common exploitation methods:

- Credential brute forcing — trying many username/password combinations automatically
- Session hijacking — stealing or forging a session token to impersonate an authenticated user

- Redirect attacks — abusing unvalidated redirects to send victims to attacker-controlled pages
- Exploiting default credentials — using factory-default usernames and passwords
- Exploiting weak credentials — guessing simple or common passwords
- Exploiting Kerberos — golden ticket, silver ticket, Kerberoasting (see Module 5)

6.1 Session Hijacking

Session hijacking occurs when an attacker steals or forges a session token to impersonate an authenticated user. A session token must be treated with the same sensitivity as login credentials — whoever holds the token controls the session.

Ways a Session Token Can Be Compromised

Method	How It Works	Defence
Predicting tokens	Guessing non-random or sequential session IDs	Use cryptographically secure random generation — minimum 256 bits of entropy
Session sniffing	Capturing unencrypted session tokens from network traffic via packet capture	Enforce HTTPS across the entire application — not just login pages
On-path (MITM) attack	Attacker sits between client and server, intercepting and manipulating web sessions or browser extensions	HTTPS with HSTS, certificate pinning, secure browser practices
Session fixation	Attacker injects a known session ID before the user authenticates — then uses it post-auth	Regenerate session ID immediately after successful authentication; use HTTPS

6.2 Redirect Attacks (Unvalidated Redirects & Forwards)

Unvalidated redirect vulnerabilities occur when an application accepts untrusted user input as a redirect destination URL. Attackers modify this URL to redirect victims to malicious sites for malware delivery or credential theft. Unvalidated forwards can also bypass internal access control checks — allowing access to privileged functions.



NOTE

Unvalidated redirect attacks typically require a small amount of social engineering — the victim must follow a crafted link that appears to come from a trusted domain.

6.3 Default Credentials

Many organisations leave infrastructure devices (routers, switches, wireless access points, firewalls, IoT devices) configured with factory-default passwords. Attackers find these devices easily using Shodan (shodan.io) and default password databases such as defaultpassword.com.

IMPORTANT

Always change default credentials during initial device setup. Restrict network management interface access to trusted management networks. A device with default credentials is effectively open to anyone who knows that device's manufacturer.

7. Exploiting Authorisation-Based Vulnerabilities

7.1 HTTP Parameter Pollution (HPP)

HTTP Parameter Pollution (HPP)

HPP vulnerabilities occur when multiple HTTP parameters share the same name in the same request. Different web servers and frameworks interpret duplicate parameters differently — some use the first value, some the last, some concatenate. Attackers exploit this inconsistency to bypass input validation, trigger application errors, or manipulate internal variable values.

```
-- Original legitimate request:
https://store.h4cker.org/?search=cars&results=20

-- Attacker appends a duplicate parameter with a different value:
https://store.h4cker.org/?search=cars&results=20&search=bikes

-- If the server uses LAST value of 'search' -> returns bikes results
-- If it uses FIRST value -> returns cars results
-- Different server behaviour = potential security bypass
```

TOOLS

OWASP ZAP is very useful for finding HPP vulnerabilities. Download: github.com/zaproxy/zaproxy

7.2 Insecure Direct Object Reference (IDOR)

Insecure Direct Object Reference (IDOR)

IDOR vulnerabilities occur when an application grants direct access to objects based on user-supplied input — without performing server-side authorisation checks to verify the user is permitted to access that specific object. Attackers modify parameter values to access other users' records, files, or account functions.

```
-- Legitimate request — accessing own customer record:
https://store.h4cker.org/buy?customerID=1188

-- Attacker modifies customerID to access a different customer's record:
https://store.h4cker.org/buy?customerID=1189

-- Legitimate request — changing own password:
https://store.h4cker.org/changepasswd?user=omar

-- Attacker modifies user parameter to change someone else's password:
https://store.h4cker.org/changepasswd?user=admin
```

TIP

IDOR Mitigations: Perform server-side authorisation checks on EVERY object access request. Use per-user or per-session indirect object references (opaque tokens) instead of exposing direct database keys.

8. Cross-Site Scripting (XSS)

Cross-Site Scripting (XSS)

XSS vulnerabilities allow attackers to inject and execute malicious scripts in web pages viewed by other users. Successful exploitation can result in: account compromise, session cookie hijacking, malware installation, local file revelation or modification, credential theft, and site redirection. XSS is one of the most common vulnerability classes in web applications worldwide.

Where XSS Vulnerabilities Typically Occur

- Search fields that echo user input back in the response
- HTTP headers that are processed and reflected by the application
- User input fields whose values appear in subsequent responses
- Error messages that return user-supplied text

- Hidden form fields containing user-supplied data
- Any page or application feature that displays user-provided content

8.1 The Three Types of XSS

Type	How It Works	Payload Location
Reflected (Non-persistent)	Malicious script is injected via a crafted link/URL. The vulnerable server reflects it back in the response. The victim must click the attacker's link. Delivery: phishing emails, messaging apps, search engines.	Not stored — in the HTTP response to the crafted request
Stored (Persistent)	Malicious script is permanently stored in the application's database (blog comments, forum posts, profile fields). Every user who views that content executes the attacker's script.	Stored on the server — delivered to ALL users who view the infected content
DOM-Based	The payload is processed entirely by the browser's DOM — never sent to the server. The attacker's malicious URL triggers DOM manipulation on the client side. The server is never aware of the attack.	Client-side only — server logs show nothing suspicious

8.2 Reflected XSS — Attack Steps

- Step 1:** Attacker discovers an XSS vulnerability in the target web server
- Step 2:** Attacker crafts a malicious link and sends it to the victim (via phishing, SMS, or social media)
- Step 3:** Victim clicks the malicious link — the attack payload is sent to the vulnerable server in the request
- Step 4:** The server reflects the malicious script back in its response — the victim's browser executes it
- Step 5:** The victim's data (cookies, credentials, page content) is exfiltrated to the attacker's server

NOTE

One of the most insidious effects of any type of XSS attack is that the victim typically does not realise an attack has taken place. The attack runs silently in the background while the victim sees a normal-looking page.

8.3 XSS Test Vectors

```
-- Basic browser address bar tests:
javascript:alert("XSS Test");
javascript:alert(document.cookie);

-- Basic form field input test:
<script>alert("XSS Test")</script>

-- Using img tag to bypass script-based filters:

<img src=javascript:alert('xss')>
<img src=javascript:alert(&quot;XSS&quot;)>

-- Using event handler attributes:
<a onmouseover="alert(document.cookie)">Hover me</a>

-- Hexadecimal HTML entity encoding (WAF evasion):
<img src=&#x6A&#x61&#x76&#x61&#x73&#x63&#x72&#x69&#x70&#x74&#x3A
&#x61&#x6C&#x65&#x72&#x74&#x28&#x27&#x58&#x53&#x53&#x27&#x29>

-- EMBED SVG tag evasion:
<EMBED SRC="..." type="image/svg+xml"
AllowScriptAccess="always"></EMBED>
```

NOTE

Attackers use obfuscation techniques to evade XSS filters and WAFs: Unicode encoding, hexadecimal HTML entity encoding, US ASCII encoding, SVG embed tags, and HTML event handler attributes. The OWASP XSS Filter Evasion Cheat Sheet documents dozens of techniques.

8.4 XSS Mitigations (OWASP Rules)

- Use an auto-escaping template system — output encoding is applied automatically
- Never insert untrusted data into the page except in explicitly allowed locations
- Apply HTML encoding before inserting data into HTML element content
- Apply attribute encoding before inserting data into HTML attributes
- Apply JavaScript encoding before inserting data into JavaScript data values
- Apply CSS encoding and strictly validate before inserting data into CSS properties
- Apply URL encoding before inserting data into HTML URL parameters
- Use the HTTPOnly cookie flag — prevents JavaScript from accessing session cookies
- Implement Content Security Policy (CSP) headers — restrict sources of executable scripts
- Use the X-XSS-Protection response header for legacy browser support
- Sanitise HTML markup using a library such as OWASP ESAPI

- Do not store sensitive information (tokens, credentials) in the DOM
- Educate users about safe browsing — avoid clicking links from unknown sources

HTML Entity Encoding — These Characters Must Always Be Encoded in Output

Raw Character	Safe HTML Entity Encoding
& (ampersand)	&
< (less than)	<
> (greater than)	>
" (double quote)	"
' (single quote)	'
/ (forward slash)	/

9. CSRF, SSRF & Clickjacking

9.1 Cross-Site Request Forgery (CSRF / XSRF)

Cross-Site Request Forgery (CSRF)

CSRF attacks transmit unauthorised commands from a user's browser — without their knowledge — to a web application that trusts them. Unlike XSS (which injects code into a page), CSRF exploits the trust an application places in an authenticated user's browser.

Example: A user is authenticated on their bank's website (session cookie in browser). They then visit an attacker's malicious page. The attacker's page silently sends a POST request to the bank to transfer funds — authenticated by the victim's existing browser cookie. The victim is unaware anything happened.

NOTE

CSRF is also called a 'one-click attack' or 'session riding'. The victim does not enter any credentials — the browser automatically includes authentication cookies in every request to the trusted domain.

REMEMBER

CSRF mitigations are implemented on the server side: Anti-CSRF tokens (synchroniser token pattern), SameSite cookie attribute (prevents cross-site cookie sending), double-submit cookie pattern, and custom request headers for API endpoints.

9.2 Server-Side Request Forgery (SSRF)

SSRF vulnerabilities allow an attacker to induce the server to make HTTP requests to an arbitrary domain or IP — including internal services normally inaccessible from the Internet. Attackers use SSRF to: scan internal network ranges, access cloud instance metadata services (AWS/GCP/Azure), read sensitive internal files, and pivot to other internal systems not exposed externally.

9.3 Clickjacking (UI Redress Attacks)

Clickjacking

Clickjacking uses transparent or opaque overlays (CSS stylesheets, iframes, and text boxes) to trick a user into clicking a button or link they cannot see. An attacker overlays an invisible iframe containing a sensitive action (e.g. 'Confirm Transfer', 'Delete Account') over something the victim intentionally clicks on. User keystrokes can also be captured this way.

Clickjacking Prevention (OWASP)

- Use the CSP frame-ancestors directive — instruct browsers not to allow the page to be framed by other domains
- Use defensive frame-busting JavaScript code to ensure the current frame is always the top-level window

10. Security Misconfigurations

10.1 Directory Traversal (Path Traversal)

Directory Traversal / Path Traversal

Directory traversal (also called dot-dot-slash, directory climbing, or backtracking) allows attackers to access files and directories stored outside the web root folder. By manipulating file

path variables using `../` sequences or absolute file paths, attackers can read sensitive system files such as `/etc/passwd`, `web.config`, `.env` files containing credentials, and private keys.

```
-- Basic path traversal - reading /etc/passwd on Linux:
https://site.com/view?file=../../etc/passwd

-- URL-encoded variant (bypasses simple filters):
https://site.com/view?file=..%2F..%2Fetc%2Fpasswd

-- Double-encoded (bypasses second layer of filtering):
https://site.com/view?file=....//....//etc/passwd

-- On Windows systems:
https://site.com/view?file=../../windows\system32\drivers\etc\hosts
```

Directory Traversal Mitigations

- Never store sensitive configuration files inside the web root directory
- Never use user-supplied input directly in filesystem or file-open calls
- Append user-supplied input to a fixed base path and resolve the canonical path before using it
- Perform strict input validation — use allow-lists of acceptable file names or paths
- Understand how the underlying OS processes file paths — different OS path separators (`/` vs `\`) require different defences

10.2 Cookie Manipulation Attacks (Stored DOM-Based)

Cookie manipulation attacks occur when a vulnerable application stores user-supplied input and embeds it in a response within the DOM. A client-side script later processes this stored input unsafely, allowing an attacker to inject arbitrary values into cookies. This can enable privilege escalation, session fixation, or persistent XSS.

TIP

Best practice: Never write data from untrusted sources directly to cookies using client-side JavaScript. Validate and sanitise all data server-side before it is stored or reflected.

11. File Inclusion Vulnerabilities

11.1 Local File Inclusion (LFI)

Local File Inclusion (LFI)

LFI vulnerabilities occur when a web application uses user-supplied input to specify which local file to include or execute — without proper validation. Successful exploitation allows reading (and sometimes executing) files on the server: sensitive configuration files, log files containing injected code, and system files. If the application runs as root, LFI can enable full system compromise.

```
-- LFI example - reading server-side files:
https://site.com/page.php?include=../../../../../etc/passwd

-- LFI via log poisoning (execute code injected into server logs):
-- Step 1: Inject PHP code into access.log via User-Agent header
-- Step 2: Include the log file via LFI to execute the code
```

11.2 Remote File Inclusion (RFI)

Remote File Inclusion (RFI)

RFI vulnerabilities allow an attacker to execute code hosted on their own remote server by supplying a remote URL as the file parameter. The vulnerable application fetches and executes the attacker's remote file. RFI is trivial to exploit — the attacker simply hosts a malicious script and provides its URL. RFI is less common than LFI but immediately gives the attacker code execution.

NOTE

RFI vulnerabilities are often enabled by misconfigured PHP settings (`allow_url_include = On`). They are less common than LFI but more dangerous when present. Always disable `allow_url_include` and `allow_url_fopen` where not required.

12. Insecure Code Practices

12.1 Comments in Source Code (CWE-615)

Developers frequently leave sensitive information in HTML, JavaScript, and CSS comments — database passwords, internal API keys, backend URLs, table names, or architectural details. An attacker who views page source (Ctrl+U in any browser) can trivially find and exploit this information.

```
<!-- TODO: Remove before production -->
<!-- DB password: Admin@123! -->
<!-- API key: sk-live-abc123xyz -->
<!-- Internal admin panel: /admin_v2_hidden/login -->
// DEBUG: username=root, password=toor
```

 NOTE

CWE-615: 'Information Exposure Through Comments.' Reference: cwe.mitre.org/data/definitions/615.html — Remove all sensitive comments before deploying to production.

12.2 Improper Error Handling

Overly verbose error messages — stack traces, SQL query strings, database error codes, and debug output — provide attackers with precise information about internal application workings. This information reveals database type, table structure, code paths, and potentially exploitable conditions.

 REMEMBER

Best practice for error handling: Display a generic, friendly error message to users. Log the full technical error server-side for developers. Provide ZERO useful technical information to potential attackers.

OWASP Error Handling Cheat Sheet:
cheatsheetseries.owasp.org/cheatsheets/Error_Handling_Cheat_Sheet.html

12.3 Hard-Coded Credentials (CWE-798)

Hard-coded credentials — passwords, API keys, database connection strings, and tokens embedded directly in source code — are catastrophic vulnerabilities. When source code is pushed to public repositories (GitHub, GitLab) or when binaries are decompiled, these credentials are exposed to anyone who finds them. Automated tools scan public repositories specifically for hard-coded credentials.

```
// Examples of hard-coded credentials — NEVER do this:
String dbPassword = "SuperSecret123!";
String apiKey = "sk-prod-abc123xyz";
String awsSecret = "wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY";

// Correct approach: load from environment variables or secrets manager
String dbPassword = System.getenv("DB_PASSWORD");
String apiKey = System.getenv("API_KEY");
```

 NOTE

CWE-798: 'Use of Hard-Coded Credentials.' Reference:
cwe.mitre.org/data/definitions/798.html — Use dedicated secrets managers (HashiCorp Vault, AWS Secrets Manager, Azure Key Vault) and environment variables instead.

12.4 Race Conditions (TOCTOU)

Race Condition / Time of Check to Time of Use (TOCTOU)

A race condition occurs when a system performs two operations that must happen in the correct sequence, but the attacker exploits the brief window between when a security check is performed (Time of Check) and when the resource is actually used (Time of Use). Race conditions have very high attack complexity but can completely bypass security controls when successfully timed and executed.

Example: A security system pushes a new firewall policy. During the brief period between the policy being compiled and being applied, an attacker can bypass the old ACLs. The window may only be milliseconds — but automated attacks can exploit it.

13. Unprotected APIs (CWE-227)

APIs are used in virtually every modern application. Many APIs lack adequate authentication, authorisation, and input validation controls. Their breadth and complexity make security testing challenging — and automated tools often miss API-specific vulnerabilities.

API Type	Data Format	Key Notes
SOAP (Simple Object Access Protocol)	XML / XSD schemas	Legacy — strict messaging patterns. Originally replaced DCOM and CORBA. Spec: w3.org/TR/soap
REST (Representational State Transfer)	JSON + OpenAPI/Swagger	Most common today — simpler than SOAP. Uses standard HTTP methods. Spec: openapis.org
GraphQL	Query language / JSON	Used in mobile apps and dashboards — allows flexible client-defined queries. Reference: graphql.org

API Penetration Testing Approach

- Collect full API requests using a proxy (Burp Suite or OWASP ZAP) — REST and SOAP APIs use more than just GET URL parameters
- Look for non-standard parameters, abnormal HTTP headers, and unexpected response fields
- Identify URL segments with repeating patterns across multiple URLs — if /s/abcd/page and /s/dead/page both exist, 'abcd'/'dead' are likely API parameters, not directories
- Inspect structured data in JSON, XML, or custom response formats for additional parameters
- Use fuzzing to find API vulnerabilities — send malformed, oversized, or specially crafted values to every parameter

API Security Best Practices

- Enforce HTTPS with strong TLS on all API endpoints — no unencrypted API services
- Validate and sanitise all parameters from API clients server-side
- Explicitly scan for common injection attack signatures — injection attacks often follow recognisable patterns
- Use strong, standards-based authentication and authorisation (OAuth 2.0, JWT with proper validation)
- Use reputable, well-maintained libraries to build and handle API interactions
- Segment API implementation and API security into distinct tiers
- Classify data sensitivity — clearly define what is publicly accessible and what requires authorisation
- Conduct API-specific code reviews with security expertise
- Require internal API documentation for all APIs in the organisation
- Never discuss company API architecture on public forums or in public code repositories

NOTE

API documentation provides a roadmap of the entire application attack surface. Always request Swagger/OpenAPI, WSDL, or WADL documentation at the start of any web application engagement.

CWE-227 (API Abuse): cwe.mitre.org/data/definitions/227.html

OWASP REST Security Cheat Sheet:

cheatsheetseries.owasp.org/cheatsheets/REST_Security_Cheat_Sheet.html

14. Hidden Form Fields, Parameter Tampering & Code Signing

14.1 Hidden Form Fields & Parameter Tampering

Web applications use hidden HTML form fields to pass data between client and server without displaying it to the user. Attackers can view these fields using browser developer tools (F12) or page source (Ctrl+U) and modify their values before form submission.

```
<!-- Hidden field in an e-commerce page — attacker can modify the price: -->
<input type="hidden" id="123" name="price" value="100.00">

<!-- Attacker edits this to: -->
<input type="hidden" id="123" name="price" value="0.01">

<!-- If the server doesn't validate the price server-side,
      the attacker purchases the item for 1 cent. -->
```

NOTE

Not all hidden form fields are vulnerabilities — some are legitimately used for application flow control and CSRF token transmission. Always test whether the server validates and enforces these values independently of what the client submits.

14.2 Code Signing & Subresource Integrity

Code signing adds a digital signature to software and applications to verify that the code has not been modified since it was signed by the developer. Applications without code signing can be modified by attackers to impersonate legitimate software.

- Code signing uses a public/private key pair to identify and authenticate the developer
- Trusted Certificate Authorities (CAs) underpin the trust chain — similar to SSL/TLS certificates
- If code is modified after signing, the public key verification fails — tampering is detected
- Subresource Integrity (SRI) extends this concept to web assets — browsers can verify that third-party scripts and stylesheets have not been tampered with using a cryptographic hash

15. Web Application Hacking & Testing Tools

Web proxies are the foundation of web application penetration testing — they sit between the browser and the server, allowing the tester to intercept, inspect, modify, and replay HTTP/HTTPS requests and responses.

Tool	Type	Description	URL
Burp Suite Community	Web proxy / scanner	Industry-standard web proxy. Intercept, modify, replay requests. Free edition available.	portswigger.net
Burp Suite Professional	Web proxy / scanner	Paid. Adds automated scanning, Intruder, Collaborator, and more.	portswigger.net
OWASP ZAP	Web proxy / fuzzer	Free, open-source. Proxy, automated scanner, fuzzer, passive scanner.	zaproxy.org
SQLmap	SQLi automation	Automated SQL injection detection and exploitation. Pre-installed in Kali.	sqlmap.org
gobuster	Directory enumeration	Fast directory and file brute-forcing written in Go.	github.com/OJ/gobuster
ffuf	Web fuzzer	Very fast web fuzzer written in Go — supports all parameters.	github.com/ffuf/ffuf
feroxbuster	Directory enumeration	Recursive content discovery written in Rust.	github.com/epi052/feroxbuster
DirBuster	Directory enumeration	Classic OWASP Java-based directory brute-forcing tool.	OWASP project
BeEF	Browser exploitation	Hooks browsers via XSS vulnerabilities and delivers browser-side attack payloads.	beefproject.com
Nikto	Web scanner	Open-source web server scanner — identifies misconfigurations,	cirt.net/Nikto2

default files, and known vulns.

TIP

PortSwigger (creators of Burp Suite) offer a free, comprehensive web application security course with interactive labs at portswigger.net/web-security — excellent preparation for PenTest+, OSCP, and other certifications.

All directory enumeration tools (gobuster, ffuf, feroxbuster, DirBuster) use wordlists — files containing words used to enumerate directories, files, and crack passwords. Common wordlists are included in Kali Linux at `/usr/share/wordlists/`

16. Module Summary & Key Takeaways

REMEMBER

Web application vulnerabilities are consistently responsible for the majority of data breaches globally. The OWASP Top 10 defines the most critical categories. Mastering web application attacks — and their mitigations — is a core skill for every penetration tester.

Key Takeaways

11. HTTP is stateless — session tokens (cookies) track authenticated users. Never include session IDs in URLs. Use HTTPS for the entire session, not just login.
12. SQL injection can read, write, delete, and destroy database data — parameterised queries (prepared statements) are the only reliable prevention.
13. The three XSS types (reflected, stored, DOM-based) all execute malicious scripts in the victim's browser — OWASP output encoding rules and CSP headers are the key defences.
14. CSRF exploits browser trust — an attacker forces authenticated requests the victim didn't intend. Anti-CSRF tokens and SameSite cookies are the primary defences.
15. IDOR allows access to other users' data by changing parameter values — server-side authorisation checks on every object reference are mandatory.
16. Directory traversal reads files outside the web root using `../` — never pass user input directly to file system calls.
17. Business logic flaws cannot be found by scanners — they require manual analysis of application workflows and intended behaviour.

18. Hard-coded credentials, verbose error messages, and source code comments all leak sensitive information that enables further attacks.
19. Race conditions (TOCTOU) create exploitable windows between security checks and resource use — high complexity but impactful.
20. API security requires testing full requests (not just URL params) via a proxy — look for missing auth, IDOR in API parameters, injection in JSON/XML body, and excessive data exposure.
21. Burp Suite and OWASP ZAP are the essential web proxy tools — master both for manual and automated web application testing.

Quick Reference: Vulnerability, Attack, and Defence

Vulnerability	Attack Technique	Key Defence
SQL Injection	UNION, Boolean, Error-based, Time delay, Stacked queries	Parameterised queries, input validation, least-privilege DB accounts
Command Injection	OS commands via unsanitised web input fields	Never pass user input to system shell; allow-list validation
LDAP Injection	Crafted LDAP queries for auth bypass or enumeration	Sanitise all input before LDAP queries; allow-list characters
XSS (Reflected)	Malicious link — script reflected from server response	Output encoding, CSP headers, HTTPOnly cookie flag
XSS (Stored)	Script stored in database — delivered to all users	Output encoding on all stored content, CSP
XSS (DOM)	Client-side DOM manipulation — server never sees payload	Avoid writing untrusted data to DOM; follow OWASP DOM XSS guide
CSRF	Forged requests using victim's authenticated browser session	Anti-CSRF tokens, SameSite cookie attribute, HTTPS
SSRF	Server makes requests to internal resources on attacker's behalf	Allow-list outbound requests; block metadata endpoints
IDOR	Modifying object reference parameters to access others' data	Server-side authorisation on every object access

HPP	Duplicate parameters with different values	Strict input validation; define clear parameter handling rules
Directory Traversal	../ sequences to read files outside web root	Allow-list path validation; never use user input in file calls
LFI / RFI	Include local/remote files via user-supplied path parameter	Disable allow_url_include; strict input validation
Clickjacking	Transparent iframe overlaying legitimate content	CSP frame-ancestors directive; frame-busting JavaScript
Business Logic	Abuse of legitimate application workflows	Threat modelling, manual code review, data validation
Hard-coded Creds	Extract credentials from source or compiled code	Secrets managers; environment variables; never embed in code

End of Module 6
